STAR OnePlatform — Build Log & Decision Record

A chronological account of how the prototype was built: the decisions made, the rationale (and alternatives weighed), the implementation at each step, and how each milestone was verified.

Span: 2026-06-08 → 2026-06-10 · **44 commits** · Repo: `github.com/rechandy/STAR-ONEPLATFORM2` · Live: `https://ruben-star-one-platform.com`

Method note. This was built with AI tools under human direction: **ChatGPT** to investigate STAR's strategy and the special-education domain, **Gemini** to structure precise prompts, **Claude Code** to do the heavy lifting in the codebase, and **Loom + ElevenLabs** to produce the walkthrough. The architecture and the corrections were human decisions; the machine wrote much of the code.

Stage 0 — Method and data model first *(Jun 8)*

Context. Before any service code, the work started with written design and a realistic data model — deliberately, because the exercise rewards *method over features*.

Decisions & rationale. - **Documents → design → code.** The first two commits are documentation, not code: the architecture *blueprint* and the *PWA-first iPad/mobile strategy*. Writing the thesis down first kept every later service consistent with one plan. - **OneRoster as the canonical model.** Rather than invent a schema, the data model follows the 1EdTech **OneRoster** standard (tenants, orgs, users, classes, enrollments, academic sessions). Rationale: it is the interoperability standard districts already speak (Clever/ClassLink export it), so identity and roster are defined once and trusted everywhere. - **Many-to-many rostering ("sections"), not 1:1.** The model was rewritten so a student belongs to *many* classes and a class has *many* staff — then layered with **co-teaching and related-service specialists** (SLP/OT/BCBA) so a student is reachable by multiple staff across schools. Rationale: this is how special education actually works (shared caseloads), and it makes the authorization model meaningful instead of trivial. - **Multi-tenant from row zero.** Every row carries `tenant_id`; Postgres Row-Level Security (`rls.sql`) is defined to enforce isolation defensively. - **Idempotent seed.** The supplied demo data (1,000 students, 4,267 IEP goals, 50 users) is ingested with deterministic ids + `createMany(skipDuplicates)` so re-running tops up without duplicating.

Implementation. - `5b2eca2` blueprint · `94121b0` client/mobile strategy. - `b1e1ee1` OneRoster Prisma schema + demo ingestion → `a378edb` many-to-many sections rewrite → `d8075a2` co-teaching + specialists → `feeef11` versioned init SQL migration. - `97b6d67` **Phase 0 scaffold:** Turborepo/pnpm monorepo, golden-path service template, PWA shell, Terraform skeleton.

Verification. Seed runs clean and idempotent; the schema captures the many-to-many graph (a sample student rostered into multiple classes; specialists spanning schools).

Stage 1 — The data spine (Phase 1) *(Jun 8)*

Context. With the model in place, the goal was the platform "spine": identity & roster reads, an outcomes store, and an event backbone — the substrate every pillar depends on.

Decisions & rationale. - **Authorization as policy (Cedar), not scattered code.** A dedicated `@oneplatform/authz` package expresses access rules as Amazon **Cedar** policies with a test suite; services evaluate `can(staff, action, student)` off the request identity. Rationale: one auditable place for "who can see/record what," re-derived identically in every service. - **Event-driven backbone via the transactional outbox.** Cross-service collaboration never uses shared tables or direct calls. A service commits its domain write **and** an `outbox` row in **one transaction**; a relay publishes to the event backbone; idempotent consumers maintain their own **CQRS** read models. Rationale: avoids dual-write inconsistency and the "distributed monolith"; gives at-least-once delivery and loose coupling. - **Config-driven broker**

(`EVENT_BROKER=memory|kafka`). The same code runs on an in-process broker for tests and **Kafka** in real runs — verified live. - **Idempotency end-to-end.** A client-generated `opId` becomes the server's `MetricEvent.idempotencyKey`, so a re-sent batch yields `duplicate`, never a double write. - **License-gated surface.** Product licenses + guardian relationships were added so the platform can show each district only what it is entitled to.

Implementation. - roster-graph: `ddb9fd4` access-set queries → `136ae9c` Cedar enforced → `d3ea2b7` `/me`, `/licenses`, admin provisioning. Policies/tests: `4f7a6a7`. - student-record: `e75de23` idempotent `/sync/mutations` → `758f733` outcome families (progress / milestone / assessment / behavior) + guarded reads. - events: `5969806` transactional outbox + `student.metric.v1` → `e0ff21e` bound to Kafka with live verification. - web: `ec3b179` offline-first PWA outbox → `23670fb` BFF round-trip to student-record. Curriculum model + IEP→objective bridge: `9a9ed24`. Offline sync protocol spec: `6f3661f`. Licenses + guardians: `248a71a`. - `93f7851` **web gateway** — login, license-filtered dashboard, admin onboarding. *(This was the checkpoint the next session resumed from.)* - `91cc98f` **SOLER** — trial-by-trial data collection emitting `student.metric.v1`.

Verification. The outbox round-trip was exercised web → student-record; the Kafka binding was verified live; Cedar policies have a passing test suite.

Stage 2 — Resume + the Phase 2 teacher loop *(Jun 9 — this session opened)*

Context. The session began with "get me back to where I left off." Reviewing memory + git established the resume point: **SOLER (Slice 1) and Links (Slice 2) were built but never live-smoke-tested** — Docker had been down.

Decisions & rationale. - **Seed the curriculum operational layer first** *(your choice).* The seed stopped at curriculum *objectives*; it didn't create the **Lessons** and **CurriculumAssignments** that Slice 2 added. Two real decisions surfaced in review: - Student-targeted assignments key on the **raw student `User.id`** (not the `sp-` profile id), because that is exactly the key the live projector and `listByStudent` match on — verified against the DB before relying on it. - **In-code de-duplication** of `(objective, student)` pairs, because a `NULL` `classId` is treated as *distinct* in the unique index, so `createMany(skipDuplicates)` would not collapse duplicates. - **Remap dev Postgres to host port 5433** *(your choice).* A host-installed PostgreSQL 18 was occupying 5432 and silently shadowing the container. Rather than stop the user's service, the dev DB was remapped to 5433 (compose + `.env.example` files), keeping the user's local Postgres intact. - **Pull endpoint on SOLER; `(updatedAt, id)` keyset cursor** *(your choices).* For the offline delta-sync, the `/sync/changes` endpoint lives on SOLER (one service for the whole offline experience), and the cursor uses the protocol's sanctioned `(updatedAt, id)` fallback — **no schema migration**.

Implementation. - `8fc88c2` seed Lessons + CurriculumAssignments (status derived from each goal's progress; one dominant class-targeted assignment per section). - `3ea60e3` Postgres → host 5433. - `0feb857` **offline delta-sync wired to SOLER:** SOLER `GET /api/sync/changes` (roster / curriculum / goals / assignments, scoped to the staff's reachable students); client IndexedDB **v2** (added `reference` + `cursors` stores) + a `pull` engine (upserts/tombstones, commit-cursor-per-page); `use-soler` hook; the SOLER station UI; BFF routes; **17 unit tests** (7 new for pull). - `9b99280` **pillar shell:** the license-gated dashboard cards became navigable links (Links → `/links`, SOLER → `/soler`); a Links curriculum page was added. - `cf54860` **fix:** a reconcile-pull after sync, so a cross-service state change reliably lands.

Bug caught in review. In the changes endpoint, spreading the keyset's `{OR}` over the access-scope `OR` **collided** and silently dropped the scope — leaking the whole tenant (3,547 assignments instead of 235). Caught because the live count didn't match the DB; fixed with `AND: [{OR: reach}, keyset]`.

Verification (the milestones). - **Live smoke test (cross-service, Kafka):** started Links (consumer) then SOLER (producer) with `EVENT_BROKER=kafka` via Redpanda; posted a session + trials + finalize; the assignment flipped **IN_PROGRESS** → **MASTERED** through outbox → Kafka → the Links projector. Confirmed the raw-`User.id` key works through both the projector and `listByStudent`. - **Real-browser walkthrough:** signed in as a teacher, captured a session offline (12 durable outbox ops), synced, and watched the assignment flip to MASTERED in the UI — which is what exposed the cross-service race fixed in `cf54860`. Verified responsive down to 375 px.

Stage 3 — Planning deliverables + the predictive model *(Jun 10)*

Context. With the loop proven, the focus moved to the exercise's executive deliverables and the integrated model.

Decisions & rationale. - **Three board-level PDFs**, then **rescoped** *(your input).* Task Plan, Budget, and Production Readiness were first drafted broadly, then re-scoped to the **four MVP capability areas** (Onboarding/Setup, Planning/Orchestration, Assignment/Progress, Admin/Leadership) and a **5-person team** (you as tech lead + 3 developers + 1 QA): ~8 months to a deployable MVP, ~$1.2M build / ~$1.85M-per-year run-rate, and an honest readiness rating (~■ to GA). - **Predictive model as a dedicated Python service** *(your choice).* Rather than bolt logistic-regression into Node, it runs as its own **FastAPI** service — the production-faithful, polyglot way to serve and retrain a model. *(Your choice)* surfaced it both on the **student record** and an **admin roll-up**.

Implementation. - `0989573` the three PDFs (reportlab generator kept for regeneration) · `136b040` `.gitattributes` marks PDFs/binaries as binary. - `120ec1a` **predict service** (`services/predict`): `features.py` (a shared train/serve feature contract + domain normalization + risk bands), `train.py` (scikit-learn `LogisticRegression`, **hold-out accuracy 0.74 / ROC-AUC 0.81**, persists `model.joblib` + an interpretable `model_card.json`), `main.py` (FastAPI scoring live from Postgres). Web integration: `/students` (caseload risk), `/students/[id]` (color-coded per-goal predictions), `/insights` (admin district roll-up), dashboard wiring.

Interventions. Stripped the Prisma-only `?schema=` from the DB URL (libpq rejects it); noticed uvicorn has no `--reload`, so the service is restarted after edits.

Verification. Bands match the spec (≥0.75 green / 0.50–0.74 yellow / <0.50 red); verified in a real browser on desktop **and** mobile, zero console errors; district distribution rendered (≈26.6% green / 19.1% yellow / 54.4% red).

Stage 4 — Deployment architecture *(Jun 10)*

Context. "We're ready to deploy" — but the proposed grouping (web + roster + ML + Postgres) omitted Kafka and the worker services.

Decisions & rationale. - **Reject serverless; deploy persistent containers** *(reinforced by analysis).* Three services (`soler`, `links`, `student-record`) run **always-on background workers** (outbox relays + Kafka consumers). Vercel/Netlify *and* AWS App Runner pause idle instances — which would stop those loops and break the very event backbone being protected. So the target is a **persistent container host**. - **Deploy the full 8-container stack** *(your choice, after the evaluation).* Anything less breaks the teacher loop, offline sync, and the event backbone. - **Reliability over image size** *(judgment call).* The NestJS image copies the built workspace wholesale (~1.2 GB) for a guaranteed-correct Prisma client, after a `pnpm deploy` slimming attempt mis-packaged the generated client. Noted as a future optimization.

Implementation. - `747f910` `infra/docker/`: one **parameterized `Dockerfile.nest`** for the four NestJS services, a Python `Dockerfile` for `predict` (model baked in), a standalone `Dockerfile.web` (added `output:'standalone'`), a one-shot `Dockerfile.migrate`, `docker-compose.prod.yml` (8 services, internal network, only `web` published, `EVENT_BROKER=kafka` for the workers, healthchecks + ordered startup), `.env.production.example`, `.dockerignore`, and the `coolify-playbook.md`. - `183e27d` a **Caddy** TLS + custom-domain layer (`docker-compose.tls.yml` + `Caddyfile`) for the public VPS deploy.

Verification. Built every image; `docker compose up --build` brought up all 8 containers healthy; migrate seeded 4,267 goals; and the **mastery flip fired through the containerized event backbone** — proving persistent server-side execution.

Stage 5 — Demo production kit *(Jun 10)*

Context. Producing the walkthrough materials and the as-built documentation.

Decisions & rationale. - **Single source of truth for narration.** The voiceover generator parses the demo-script markdown directly, so the script stays the one editable source. - **Two voices.** A first-person **Introduction & Method** (the presenter's own voice) precedes a third-person, James-Earl-Jones-register narration over the demo body. - **Diagrams-as-code.** The architecture diagrams are **Mermaid** (render on GitHub, version-controlled), rasterized for the PDF via a hosted renderer.

Implementation. - `14b1e03` synchronized Loom demo script → `8a51fbb` ElevenLabs narration generator → `6003e2c` Introduction & Method + deepened the predictive-model scene → `6477c06` truststore TLS + `imageio-ffmpeg` unified-track encoding → `ac986ea` strip markdown before TTS (+ `a6d9744` gitignore generated audio). Result: an **8:22** deep-baritone ("Brian") narration track + per-scene clips. - `f970024` **as-built architecture doc + Mermaid diagrams** → `d5c939f` PDF export (mermaid.ink + `markdown` + `xhtml2pdf`) → `7c2d1d7` printable landscape demo-script PDF → `bb4dbe0` generalized the md→PDF converter (`--src/--out/--landscape`).

Interventions (environment). TLS verification failed behind a corporate proxy → routed the SDK through the **OS trust store** (truststore). No system `ffmpeg` and Python 3.14 dropped `audioop` → used the pip-bundled `imageio-ffmpeg` binary to produce a valid unified MP3.

Stage 6 — Live public deployment *(Jun 10)*

Context. The platform needed a public URL for stakeholders and the Loom.

Decisions & rationale. - **Tunnel first, then real infra.** A no-account **Cloudflare quick tunnel** over the local stack gave an instant public HTTPS URL — but it dropped twice (an Error 1033 + a Docker port hiccup), fragile on the proxied network. That motivated the move to durable hosting. - **Register a domain + VPS** *(your choice).* `ruben-star-one-platform.com` registered on Cloudflare (DNS + TLS unified) + a DigitalOcean droplet (Ubuntu 24.04, 2 vCPU / 8 GB). Confirmed availability via RDAP first. - **DNS-only + Caddy/Let's Encrypt** *(judgment call).* The A record is proxy-off (grey cloud) so Caddy can complete the HTTP-01 challenge directly and serve a valid Let's Encrypt cert.

Implementation & verification. - SSH from the local machine reached the VPS (port 22 not blocked). Installed Docker, cloned the repo, built + started all 8 containers (seeded 4,267 goals), and verified the app on `http://<ip>:3000`. - Pulled the Caddy TLS layer; you added the DNS A record; Caddy obtained the LE cert; **`https://ruben-star-one-platform.com`** went live with HTTP→HTTPS redirect. - **Health-verified:** 20/20 requests 200 (≤0.18 s), every page 200, 0 container restarts, no OOM, Kafka consumers connected, valid TLS through Sep 8 — and reachable from the user's own network. The redundant local stack + tunnel were then stopped.

Load-bearing decisions, at a glance

Decision	Choice	Why
Build method	Documents → design → code	Method over features; consistency
Canonical model	OneRoster, many-to-many	Real co-teacher / specialist rostering; interop
Tenancy	`tenant_id` everywhere + RLS	Defensive multi-tenant isolation
Service collaboration	Transactional outbox → Kafka → CQRS	No shared DB; at-least-once; decoupled
Authorization	Cedar policies + tests	Policy, not scattered conditionals

Idempotency	`opId` → `MetricEvent.idempotencyKey`	Safe retries; exactly-once effect
Client	Offline-first PWA (IndexedDB)	Classrooms have unreliable Wi-Fi
Delta-sync cursor	`(updatedAt, id)` keyset	Stable paging, no schema change
Predictive model	Dedicated Python (FastAPI) service	Polyglot; real ML serving + retraining path
Hosting	Persistent containers (VPS/Coolify)	Serverless pauses the always-on workers
Prod TLS/domain	Caddy + Let's Encrypt, DNS-only	Automatic HTTPS; reproducible
Image build	Copy built workspace	Reliable Prisma client over smaller images

---

Verification milestones (how we knew it worked)

1. Seed runs idempotently into the OneRoster graph (Stage 0/2).
2. Outbox → Kafka round-trip verified live (Stage 1).
3. **Cross-service mastery flip** over Kafka — CLI smoke test (Stage 2).
4. **Full teacher loop in a real browser**, incl. offline capture + sync (Stage 2).
5. Predictive model scored live, color-coded, desktop + mobile (Stage 3).
6. **8 containers healthy + mastery flip in containers** (Stage 4).
7. **Live public site** stress-tested healthy on a real domain with valid TLS (Stage 6).

Appendix — full commit history (oldest first)

Date	Commit	Summary
06-08	5b2eca2	docs: foundational architecture blueprint
06-08	94121b0	docs: PWA-first iPad/mobile client strategy
06-08	b1e1ee1	feat(database): OneRoster Prisma schema + demo ingestion
06-08	a378edb	feat(database): many-to-many sections rewrite
06-08	d8075a2	feat(database): co-teaching + related-service specialists
06-08	feeef11	feat(database): versioned init SQL migration

06-08	97b6d67	feat(monorepo): Phase 0 skeleton (Turborepo, template, PWA, Terraform)
06-08	ddb9fd4	feat(roster-graph): access-set queries over seeded data
06-08	4f7a6a7	feat(authz): Cedar student-access policies + tests
06-08	9a9ed24	feat(database): Curriculum model; IEP goals → Links objectives
06-08	6f3661f	docs: on-device offline sync protocol spec
06-08	136ae9c	feat(roster-graph): enforce Cedar authorization
06-08	e75de23	feat(student-record): authorized, idempotent /sync/mutations
06-08	758f733	feat(student-record): outcome families + guarded reads
06-08	5969806	feat(events): transactional outbox + event backbone
06-08	ec3b179	feat(web): offline-first PWA outbox wired to /sync/mutations
06-08	e0ff21e	feat(events): bind event backbone to Kafka (verified)
06-08	23670fb	feat(web): BFF route for the outbox round-trip
06-08	248a71a	feat(database): product licenses + guardian relationships
06-08	d3ea2b7	feat(roster-graph): /me, /licenses, admin provisioning
06-08	93f7851	feat(web): gateway — login, dashboard, onboarding
06-08	91cc98f	feat(soler): trial-by-trial data collection
06-09	1991411	feat(links): scope/sequence, assignments, adapt consumer
06-09	8fc88c2	feat(database): seed Links lessons + assignments

06-09	3ea60e3	chore(database): remap dev Postgres to 5433
06-09	0feb857	feat(soler+web): offline-first delta sync
06-09	9b99280	feat(web): pillar shell — open Links & SOLER
06-09	cf54860	fix(web): reconcile pull after sync
06-09	7af2303	chore: preview launch config
06-10	0989573	docs: prototype-to-production MVP planning deliverables
06-10	136b040	chore: mark PDFs/binaries as binary
06-10	120ec1a	feat(predict): IEP goal-attainment model + risk UI
06-10	747f910	feat(deploy): containerized full-stack deployment
06-10	14b1e03	docs: synchronized Loom demo script
06-10	8a51fbb	feat(demo): ElevenLabs narration generator
06-10	6003e2c	docs(demo): Introduction & Method; deepen model scene
06-10	6477c06	fix(demo): robust TLS + unified-track encoding
06-10	a6d9744	chore: gitignore generated demo audio
06-10	ac986ea	fix(demo): strip markdown before TTS
06-10	f970024	docs(architecture): as-built prototype architecture + diagrams
06-10	d5c939f	docs(architecture): PDF export of the architecture doc
06-10	7c2d1d7	docs(demo): printable landscape PDF of the demo script
06-10	bb4dbe0	chore(docs): generalize md→pdf converter
06-10	183e27d	feat(deploy): Caddy TLS + custom-domain layer